

Background: Introduction

Highlights

- Brief history and roots of HCI and UX.
- Shifting paradigms in HCI and UX.
- Fun interaction at work.
- Other topics related to phenomenology.
- Who introduced the waterfall model?
- Silos, walls, and boundaries.

6.1 THIS IS A REFERENCE CHAPTER

This chapter contains reference material relevant to the other chapters of [Part 1](#). This chapter is not intended to be read through as a regular chapter, but each section is supposed to be read when a reference to it in the main chapters is encountered.

6.2 BRIEF HISTORY AND ROOTS OF HCI AND UX

This is a terse capsule of the history and roots of HCI and UX. The coverage is by no means a survey of the vast contributions in this large field.

It is a matter of debate exactly when computer usability was born. We know that it was a topic of interest to some by the late 1970s, and by the early 1980s, conferences about the topic were being established. Although work was being done on “human factors in computers” in the 1970s and earlier, HCI was emerging at various universities in the late 1970s and 1980s and had been going on at IBM ([Branscomb, 1981](#)), the National Bureau of Standards (now the National Institute of Standards and Technology), and other scattered locations before that.

Human Factors

An engineering discipline dedicated to bringing science and technology together with human behavior and biological characteristics for design and maintenance of products and systems for safe, effective, and satisfying use (Section 6.3.4).

Many believe that HCI did not coalesce into a fledgling discipline until the first CHI conference (Conference on Human Factors in Computer Systems) in Boston in 1983. But it probably began at least a couple of years before with the “unofficial first CHI conferences” (Borman & Janda, 1986) at the May 1981 ACM/SIGSOC conference, called the Conference on Easier and More Productive Use of Computer Systems, in Ann Arbor, Michigan, and the March 1982 Conference on Human Factors in Computer Systems in Gaithersburg, Maryland.

Human-computer interaction in general and usability and UX in particular owe much of their origin and development to influences from many other related fields.

6.2.1 Frederick Winslow Taylor: Scientific Management

On a timeline, our story begins very early, more than a century ago, with the advent of a concept called scientific management. Sometimes called “Taylorism,” this method was created by Frederick Winslow Taylor, a mechanical engineer. He is also known for helping formulate a national (in this case, the United States) imperative for increased industrial efficiency. Taylor sought to define “best practices” of the time to reform our inefficient and wasteful, even lazy, ways of operating private and government enterprises and factories (Taylor, 1911).

Taylor came up with two tenets to increase the productivity of our workforce, tenets that remained popular for the next 40–50 years:

- Rather than trying to find the right person for a task (i.e., someone who already has the knowledge and skills for the job), we must train the person to fit the task.
- The system rather than the person must come first.

With more than 100 years of hindsight, we can now see that he pretty much got it backwards but, at the time, it was an important idea for engineering the workplace.

6.2.2 Early Industrial and Human Factors Engineering

WWI: Designing the human to fit the machine. By the time we were involved in World War I from roughly 1914–18, Taylor’s ideas for industrial engineering were in full stride. Training was used to fit the human to work with the machine. Engineers built aircraft with all their controls and displays and then brought in potential pilots and trained them to fit the designs.

WWII: Birth of human factors and ergonomics. By the time we were engaged in World War II, the notion that you can train any human to fit any design was showing

some cracks, which became substantial enough to start a new discipline, called human factors and ergonomics.

Col. Fitts and Cpt. Jones. The early heroes in this new field included Col. Paul Fitts and Capt. Richard Jones. US Air Force officials were concerned about airplane crashes suffered by experienced World War II pilots. Very fit, highly trained, and highly motivated people were making significant errors in task performance in the field. No matter how extensive the training or flying experience, pilots were making dangerous mistakes while operating the controls in the cockpit. Well-trained pilots were failing to detect the enemy on radars and were making “pilot errors” in airplane cockpits. [Fitts and Jones \(1947\)](#) concluded that the cause could not be flaws in training.

So, they studied critical incidents that may have led to airplane crashes. To better understand what was going on, they started interviewing pilots to see what problems they had, a practice that lives on today.

BT-13 airplane. In some of these interviews, Fitts and Jones talked with pilots of the BT-13 aircraft, who described unsettling scenarios they encountered during take off. As the plane was rolling down the runway, the pilot wanted to adjust the pitch of the prop to give more thrust but instead “accidentally” adjusted the fuel mixture, causing a loss in power and the need to abort the takeoff.

Instead of pulling the prop pitch lever, the pilot was pulling the fuel mixture lever. Why? The two levers were right next to each other and looked exactly like one another. Therefore, it was easy to make a slip and grab the wrong lever, an act they came to understand could not be avoided by any amount of training.

B-17 airplane. In similar interviews, pilots of the B-17 bomber related comparable stories about landing the plane. In one such story heard frequently, as they came in for a landing the pilot asked the copilot to turn on the landing lights. But somehow the copilot “accidentally” hit the flaps switch. The plane nosed into the runway and suffered serious damage. Fortunately, the pilot and copilot survived and, therefore, could report the incident.

The reports of Fitts and Jones are among the very earliest that recognized the causal connection between design flaws, rather than human errors, and mishaps in user performance.

Lt. Alphonse Chapanis. The third hero of this era was Lt. Alphonse Chapanis, a human factors researcher called in to investigate another peculiar behavior: pilots of B-17s, B-25s, and P-47s would retract the landing gear after landing! Why were these people doing such a foolish thing? And why was that happening with just these aircraft and not others? Through observations and pilot interviews, they discovered that the controls for retracting the landing gear and the controls for adjusting the flaps after landing were, you guessed it, similar in appearance and located right next to each other.

Cognitive Affordance

A design feature that helps users with their cognitive actions: thinking, deciding, learning, understanding, remembering, and knowing about things (Section 30.2).

Chapanis realized that he couldn't change the design of the layout of controls in these mass-produced aircraft, so he came up with a clever in-the-field fix. He put two different kinds of knobs on these controls—one was a circular object much like the wheel from a rolling chair and the other was a wedge-shaped solid object. Then, through tactile feedback without even looking, a pilot can feel the circular, wheel-like object and know that's for the wheels. When the pilot felt the wedge-shaped handle, he knew that's for the flaps.

This must have been one of the very earliest examples of redesign to meet user needs, adding cognitive affordances through an understanding of how users were using a design in a way that was different from the designers' original intent.

6.2.3 Dreyfuss, after WW II

In the mid-1950s, in his seminal book *Designing for People*, Henry Dreyfuss, an American industrial designer, wrote that difficulties in interaction with a product are not necessarily the fault of the human user, but often it is because the designer has failed.

6.2.4 Human Factors Meets HCI

Human factors is about making things work better for people. For example, think about building a bridge: You use theory, good design practice, and engineering principles, but you can't really know if it will work. So you build it, but who's going to be the first one to test it? Well, that's one of the reasons we have graduate students.

— Phyllis Reisner

Human factors is an engineering discipline dedicated to bringing science and technology together with human behavior and biological characteristics for design and maintenance of products and systems for safe, effective, and satisfying use. It's not surprising that, when human factors entered the computer age, it made a good fit with the emerging field of human-computer interaction and usability.

In fact, many ideas and concepts from human factors laid the basis for HCI techniques later on. For example, the idea of task analysis was first used by human factors specialists in analyzing factory workers' actions on an assembly line. For many working in human factors engineering, the move to focus on HCI was a natural and easy transition.

Psychology and cognitive science. Much of the foundation for HCI has also been closely related to theory in psychology, derived from adaptations of psychological theory and models of human information processing (HIP) (Barnard, 1993; Hammond, Gardiner, & Christie, 1987). Concepts such as user modeling and

user performance metrics were adopted into HCI from cognitive and behavioral psychology and psychometrics.

Perhaps the most important application of psychology to HCI has been in the area of modeling users as human information processors (Moran, 1981b; Williges, 1982), which offered the first theory within HCI. Most human performance prediction models stem from Card, Moran, and Newell (1983), including the keystroke level model (Card, Moran, & Newell, 1980), the command language grammar (Moran, 1981a), the Goals, Operators, Methods, and Selections (GOMS) family of models (Card et al., 1983), the cognitive complexity theory of Kieras and Polson (1985), and programmable user models (Young, Green, & Simon, 1989).

HCI was also influenced by the empirical aspects of psychology. For example, Fitts law (relating cursor travel time to distance and size of target) (Fitts, 1954; MacKenzie, 1992) is clearly connected to kinesthetics and human performance.

Like human factors engineering, cognitive psychology has many connections to design for human performance, including cognition, memory, perception, attention, sense and decision making, and human behavioral characteristics and limitations, elements that clearly have much to do with user experience.

6.2.5 Computer Science: Hardware and Software Foundations of Human-Computer Interaction

In the 1960s, more people with less technical training were getting into contact with computers. Up until this time, all user interaction was keyboard based—until the very first experimental pointing device, the first mouse (Fig. 6-1), invented by Douglas Engelbart.¹

At this time, interaction devices were not standardized; you couldn't take an input device from one computer and use it on the next. So a new focus in computer science emerged in creating devices, interaction styles, and supporting software for interoperability and for human ergonomics. Soon (c.1960), we were in an era of research and development on keyboards, CRT terminals, text editors, and training manuals. At this point, the main participants in HCI from computer science were software people and much of the work was about input/output devices and user interface programming.

Software engineering. The closest kin of human-computer interaction and usability engineering on the computer science side is the more mature discipline of software engineering. Despite very different philosophical underpinnings, the

Human Information Processing (HIP)

An approach to HCI based on the cognitive science metaphor of “mind and computer as symmetrically coupled information processors” (Harrison, Tatar, & Sengers, 2007) (Section 6.4.2).

¹https://en.wikipedia.org/wiki/Computer_mouse.



Fig. 6-1

*The first computer mouse
(from SRI International).*

two fields have development lifecycles with similar and complementary structures, including activities such as requirement gathering, design, and evaluation.

In an ideal world, one would expect close connections between these two lifecycles as they operate in parallel during the development of a unified interactive system. However, in the past, these two roles typically have not communicated with one another much until the very end when actual implementation starts. This was often too late, especially when interaction design concerns have serious software architectural implications. One of the reasons for this apparent lack of connection between the two lifecycles is because of how these two disciplines grew, without either one strongly influencing the other. In fact, barring a few exceptions, the software engineering and usability engineering researchers and practitioners have mostly ignored one another over the years. Fortunately, this is changing in today's agile approaches.

Interactive graphics, devices, and interaction techniques. A new thread began appearing in the literature and practice on the computer science side of the HCI equation. This work on interactive graphics, interaction styles, software tools, dialogue management systems, programming language translation, and interface “widgets” was essential in opening the way to practical programming techniques for bringing interaction designs to life on computers.

The origin of computer graphics is frequently attributed to pioneers such as [Ivan Sutherland \(1963, 1964\)](#) and solidified by masters such as [Foley](#) and colleagues ([Foley & Van Dam, 1982](#); [Foley, Van Dam, Feiner, & Hughes, 1990](#); [Foley & Wallace, 1974](#)) and [Newman \(1968\)](#). For an insightful account of the relationship of graphics to HCI, see [Grudin \(2006\)](#).

The 1980s and 1990s saw a burgeoning of hardware and software development to support the now familiar point-and-click style of interaction, including the Xerox Star (Smith, Irby, Kimball, Verplank, & Harslem, 1989) and the Lisa and Macintosh by Apple. Personal computing brought with it a kind of democratization of computing—computing for everyone, not just the elite geeks.

In the context of an interaction technique (Foley et al., 1990), an interaction object and its supporting software were often referred to as a “widget.” Programmers developed libraries of widget software to support programming of graphical user interfaces. Early graphics packages took interaction beyond text to direct manipulation of graphical objects, eventually leading to new concepts in displays and cursor tracking. No longer tied to just a keyboard or even just a keyboard and mouse, many unusual (then, and some still now) interaction techniques arose (Buxton, 1986; Hinckley, Pausch, Goble, & Kassell, 1994; Jacob, 1993). Myers led the field in user interface software tools of all kinds (Myers, 1989, 1992, 1993, 1995; Myers, Hudson, & Pausch, 2000), and Olsen is known for his work in user interface software (Olsen, 1983).

So many people contributed to the work on User Interface Management Systems (UIMS) that it is impossible to even begin to recognize them all. Buxton, Lamb, Sherman, and Smith (1983) were among the earliest thinkers in this area. Others we remember are Brad Myers, Dan Olsen, Mark Green, Jim Foley’s group at GWU, and our researchers at Virginia Tech. Much of this kind of work was, and still is, reported in the ACM Symposium on User Interface Software and Technology (UIST), a conference specifically for user-interface software.

The commercial world followed suit and we worked through quite a number of proposed “standard” interaction styles, such as OSF Motif (The Open Group). Developers had to choose from those available mainly because the interaction styles were not interoperable. Each approach was tied closely to its own software tools for generating the programming code for interaction designs using the devices and interaction styles of these approaches. Standardization led to today’s GUI platforms and corresponding styles.

This growth of graphics and devices made possible one of the major breakthroughs in interaction styles—direct manipulation (Hutchins, Hollan, & Norman, 1986; Shneiderman, 1983), changing the basic paradigm of interaction with computers. Direct manipulation allows opportunistic and incremental task planning. Users can try something and see what happens, exploring many avenues for interactive problem solving.

6.2.6 Changing Concepts of Computing and Interaction

For a long time, interaction was about screens full of data fields displayed on dumb terminals connected to mainframes. When PCs first came along, their screens pretty much looked the same—full of panels containing data fields.

Paradigm

A model, pattern, template, or intellectual perception or view guiding a way of thinking and doing. Historically, with respect to a field of thought and work, it is thought of as coming in waves over time (Section 6.4).

Interaction (in UX)

A broad term referring to a wide variety of communication and collaboration between a human and a device, product, or system in an ecology (Section 1.1).

Then along came GUIs, the Internet, and the web. And now it is going well beyond that to all kinds of personal, handheld, and mobile devices as well as devices embedded in appliances, homes, and cars—and beyond.

Sitting in front of a desktop or laptop usually conveys a feeling of “doing computing” to users. But, when we drive a car we don’t think of ourselves as “doing computing,” yet we are using the car’s built-in computer and maybe even a GPS. As [Mark Weiser \(1991\)](#) said, “the world is not a desktop.”

6.2.6.1 *Disappearing technology*

Interaction, however, is doing more than just reappearing in different devices such as we see in web access via mobile phone. [Weiser \(1991\)](#) said “... the most profound technologies are those that disappear.” [Russell, Streitz, and Winograd \(2005\)](#) also talk about the disappearing computer—not computers that are ceasing to exist, but disappearing in the sense of becoming unobtrusive and unremarkable. They use the example of electric motors, which are part of many machines we use daily yet we almost never think about electric motors per se. They talk about “making computers disappear into the walls and interstices of our living and working spaces.” The next two sections are about kinds of interaction that can occur in this disappearing technology.

6.2.6.2 *Embedded, ubiquitous, and ambient interaction*

Embedded interaction, ubiquitous interaction, and ambient interaction are somewhat related concepts having to do with the relationship of devices and systems as well as their interaction with their environments. The terms are very similar and there is much overlap.

Embedded interaction. *An embedded system is a computational system (a computer-like device) set within another device or system—for example, a control unit embedded within a home appliance. We can walk around with wearable computing devices embedded in our clothing or within a shoe. In a project at MIT, volunteer soldiers were instrumented with sensors that could be worn as part of their clothing to monitor heart rate, body temperature, and other parameters to detect the onset of hypothermia* ([Zieniewicz, Johnson, Wong, & Flatt, 2002](#)).

Practical applications in business already reveal the almost unlimited potential for commercial applications. [Gershman and Fano \(2005\)](#) cite an example of a smart railcar that can keep track of and report on its own location, state of repair, whether it is loaded or empty, and its routing, billing, and security status (including aspects affecting homeland security). Imagine the promise this shows for improved efficiency and cost savings over the mostly manual and error-prone methods currently used to keep track of railroad cars.

In fact, you can embed radio frequency identification (RFID) chips and even GPS capabilities into almost any physical object and connect it wirelessly to the Internet. Such an object can be queried about what it is, where it is, and what it is doing. You can ask your lost possessions where they are (Churchill, 2009; Gellersen, 2005). There are obvious applications to products on store or warehouse shelves and inventory management. More intelligence can be built into these objects, giving them capabilities beyond self-identification to sensing their own environmental conditions.

Ubiquitous interaction. As the term “ubiquitous” implies, ubiquitous interaction technology—often called ubiquitous computing (Weiser, 1991) in the literature—can exist almost anywhere. Interaction technology can reside in appliances, homes, offices, stereos and entertainment systems, vehicles, roads, and objects we carry (briefcases, purses, wallets, wrist watches). This interaction in everyday contexts is taking place without keyboards, mice, or monitors. As Cooper, Reimann, and Dubberly (2003) say, you do not need a traditional user interface to have interaction.

Kuniavsky (2003) concludes that ubiquitous computing requires extra careful attention to design for the user experience. He believes ubiquitous computing devices should be narrow and specifically targeted rather than multipurpose or general-purpose devices looking more like underpowered laptops. And he emphasizes the ecological perspective: We need to design complete systems and infrastructures rather than just devices.

An example of distributing the “system” into the environment is the “Amazon Dash” order buttons. These are buttons you have around the house that, when pressed, will place a preconfigured order. So an Amazon Dash button next to a washing machine in the laundry room can be pressed to order detergent when you are running low. That is the only capability of the button. These buttons also leverage tangible and embodied interaction advantages because of the physicality of pressing the button to order a product.

Ambient interaction. As the term “ambient” implies, ambient interaction technology is “environmental” in that it is in everyday surroundings and, as such, it can also be thought of as embedded and ubiquitous.

Your house, its walls, and its furniture can surround you with interactive technology. An ambient system can extract or sense inputs from the environment and take the initiative to communicate with humans and with other objects and devices (Tungare et al., 2006) without a deliberate or conscious action by a user. The simplest example is a thermostat that senses the ambient temperature and adjusts the heat or cooling accordingly. As a more sophisticated example, a “smart wall” can proactively extract inputs it needs by sensing a user’s presence

and identifying the user with something like RFID technology. It is still user-system interaction, only the system is controlling the interaction and the human “user” may not always be aware, or need to be aware, of the interaction.

This kind of interaction is sometimes called “ambient intelligence,” the goal of considerable research and development aimed at the home living environment. In the HomeLab of Philips Research in the Netherlands (Markopoulos, Ruyter, Privender, & Breemen, 2005), researchers believe “that ambient intelligence technology will mediate, permeate, and become an inseparable common (element) of our everyday social interactions at work or at leisure.”

Everyday objects such as milk and groceries can be tagged with inexpensive machine-readable identifiers, allowing changes in those artifacts to be detected automatically. This means your mobile phone can keep in touch with your refrigerator to track items you need (Ye & Qiu, 2003). And, as you approach the grocery store, you get a reminder to stop and pick up some milk.

More and more applications that were in research labs are now moving into commercial adoption. For example, robots in more specialized applications than just housecleaning or babysitting are gaining in numbers (Scholtz, 2005). There are robotic applications for healthcare rehabilitation, including systems to encourage severely disabled children to interact with their environment (Lathan, Brisben, & Safos, 2005); robotic products to assist the elderly (Forlizzi, 2005); robots as laboratory hosts and museum docents (Sidner & Lee, 2005); robot devices for urban search and rescue (Murphy, 2005); and, of course, robotic rover vehicles for unmanned space missions (Hamner, Lotter, Nourbakhsh, & Shelly, 2005).

6.2.6.3 *Situated, embodied, and tangible interaction*

Situated interaction. While embedded, ubiquitous, and ambient technologies take something of an outside-in ecocentric perspective, as in how the environment relates to the user, situated, embodied, and tangible interactions take a somewhat user-centric inside-out perspective, as in how the user relates to the environment.

Situated awareness refers to technology that is aware of its context. As an example, this includes awareness of the presence of human users in an activity space. In a social interaction setting, this can help find specific people, help optimize deployment of a team, or help cultivate a feeling of community and belonging (Sellen, Eardley, Izadi, & Harper, 2006).

Being situated is all about a sense of “place,” the place of interaction within the broader usage context. An example of situated awareness (credit not ours) is a cell phone that “knows” it is in a movie theater or that the owner is in a nonphone

conversation; that is, the device or product encompasses knowledge of the rules of human social politeness.

Embodied and tangible interaction. Complementing situated awareness, *embodied interaction* refers to the ability to involve one's physical body in interaction with technology in a natural way, such as by gestures. Antle (2009) defines embodiment as, "how the nature of a living entity's cognition is shaped by the form of its physical manifestation in the world." As she points out, in contrast to the human as information processor view of cognition, humans are primarily active agents, not just "disembodied symbol processors." This implies bringing interaction into the human's physical world to involve the human's own physical being.

Embodied interaction, first identified by Malcolm McCullough in *Digital Ground* (McCullough, 2004) and further developed by Paul Dourish in *Where the Action is* (Dourish, 2001), is central to the changing nature of interaction. Dourish says, "how we understand the world, ourselves, and interaction comes from our location in a physical and social world of embodied factors." Embodied interaction is action situated in the world.

To make it a bit less abstract, think of a person who has just purchased something with "some assembly required." To sit with the instruction manual and just think about it pales in comparison to physically doing the assembly—holding the pieces and moving them around, trying to fit them this way and that, seeing and feeling the spatial relations and associations among the pieces, seeing the assembly take form, and feeling how each new piece fits. This is exactly the reason that physical sketching gives such a boost to invention and ideation. The involvement of the physical body, the motor movements, the visual connections, and the potentiation of hand-eye-mind collaboration lead to an embodied cognition far more effective than just sitting and thinking.

Although tangible interaction seems to have a following of its own (Ishii & Ullmer, 1997), it is very closely related to embodied interaction. You could say that they complement each other. *Tangible interaction involves physical actions between human users and physical objects.* Industrial designers have been dealing with it for years, designing objects and products to be held, felt, and manipulated by humans. The difference now is that the object involves some kind of computation. And there is a strong emphasis on physicality, form, and tactile interaction (Baskinger & Gross, 2010).

More than ever before, tangible and embodied interaction calls for physical prototypes as three-dimensional sketches to inspire the ideation and design process.

Ideation

An active, creative, exploratory, highly iterative, fast-moving, and usually collaborative, brainstorming process for forming ideas for design (Section 14.2).

Sketching

The rapid creation of free-hand drawings expressing preliminary design ideas, focusing on concepts rather than details. Is an essential part of ideation. A sketch is a conversation between the sketcher or designer and the artifact (Section 14.3).

6.2.7 Evolving Importance of UX

6.2.7.1 *Emerging desire for usability*

Initial tolerance of a poor user experience. In the distant past, computer usage was esoteric, conducted mostly by a core of technically oriented users who were not only willing to accept the challenge of overcoming poor usability, but who sometimes welcomed it as a barrier to protect the craft from uninitiated “outsiders.” Poor usability was good for the mystique, not to mention job security. And for other users, being new to this incomprehensible technology meant acceptance of the learning and frustration ahead.

Dancing bear software. Sometimes, even more recently, we have what Cooper (2004, p. 26) calls “dancing bear” software. It is where an amazing idea triumphs over poor design. It is about having features just so good users cannot do without it, even if it has a terrible interaction design. Just having a bear that can dance leads one to overlook the fact that it cannot dance very well. Success despite poor interaction design can be used as a justification for resisting change and keeping the bad design ideas: “We have been doing it that way, our product is selling phenomenally, and our users love it.” Think of how much better it could be with a good design.

6.2.7.2 *The rise of usability engineering*

Roughly in the 1990s, the focus moved to usability engineering, which focused heavily on elaborate usability evaluation methods. It has been more than two decades since the first usability engineering “process” books, such as Nielsen (1993), Hix and Hartson (1993), and Mayhew (1999), emerged from this period. This era encompassed early human-computer interaction (HCI) concepts, but still had a bit of a software flavor (e.g., in the software tools for user interface implementation) as well as “standards” for look and feel. This era of the discipline did much to improve usability of software everywhere and to raise awareness of the need to do so.

6.2.7.3 *The rise of user experience*

Up until about 1990, the focus had been mostly on methods—how can we come up with a prescriptive process that teams can use to realize some level of usability in the final product, usually in a business or professional setting?

But computers, especially personal computers, were becoming more affordable to individuals and it was more important for the average person to attain a good overall user experience in their use. Sometime about 2000, voices like that of Don Norman moved the focus from the narrow concept of usability to a broader idea of the user experience (UX), embracing more of the total usage phenomenon and including emotional impact. And that’s where we are with this book!

6.3 SHIFTING PARADIGMS IN HCI AND UX

A paradigm is a model, pattern, template, or intellectual perception or view guiding a way of thinking and doing. Historically, with respect to a field of thought and work, thought of as coming in waves over time.

Harrison et al. (2007) tell us that designers approach design problems with a particular worldview, embracing specific practices, expectations, and values. They call these worldview paradigms, models of thinking that come as waves of thought in time.

Some of the history of HCI and UX can be seen as “waves” of these paradigms, shifting over time. About the same time, Susanne Bødker introduced her own concept of a third wave of HCI in her keynote address at NordCHI 2006 (Bødker, 2015).

Harrison et al. (2007) distinguish three major intellectual waves that have formed the field of HCI:

- Engineering—human factors engineering and usability engineering: To optimize the match between human and machine. The metaphor of interaction is about a match of human and machine.
- Human information-processing model and cognitive science: This wave emphasized models of the relationship of the human mind to computers and theories of what is happening in the human mind during and with respect to interaction. The metaphor of interaction is “human minds are like information processors.”
- Phenomenology: This wave focuses on the experiential quality of interaction. The metaphor of interaction is about making meaning (more on this when we talk about phenomenology) and how users experience meaning within an artifact and its use.

The driving force of their third paradigm was the fact that the essence of social and situated actions was at odds with both the usability-oriented engineering paradigm and the cognitive logic of the human information processor approach. Given the initial reluctance of HCI as a field to embrace the phenomenological paradigm, the Harrison et al. paper was an evangelical wake-up call to pay attention and include the phenomenological paradigm in mainstream HCI.

6.3.1 Engineering Paradigm

With its roots in software and human factors engineering, the engineering paradigm in HCI prescribed starting with an inventory of the functionality envisioned for a new system and proceeding to build those items with the best quality possible given available resources. The engineering focus is on

Human Factors

An engineering discipline dedicated to bringing science and technology together with human behavior and biological characteristics for design and maintenance of products and systems for safe, effective, and satisfying use (Section 6.3.4).

functionality, reliability, user performance, and avoiding errors. Recognizing that user interaction deserved attention on its own, usability engineering emerged as a practical approach to usability with a focus on improving user performance, mainly through evaluation and iteration.

The engineering paradigm also had strong roots in human factors, where "work" was studied, deconstructed, and modeled. An example is the study of an assembly line where each action required to do work was carefully described. It was a purely utilitarian and requirements-driven approach. Alternative methods and designs were compared and success was measured by how much the user could accomplish.

6.3.2 Human Information Processing (HIP) Paradigm

The HIP approach to HCI is based on the cognitive science metaphor of "mind and computer as symmetrically coupled information processors." This paradigm at its base is about models of how information is sensed, accessed, and transformed in the human mind and, in turn, how those models reflect requirements for the computer side of the information processing. It was defined by Card et al. (1983) and well explained by Williges (1982).

6.3.3 Phenomenological Paradigm

Harrison, Tatar, and Sengers called their third HCI design paradigm the "phenomenological matrix" (Harrison et al., 2007).

Phenomenological aspects of interaction are part of what was called the "hedonic phase" (the rise of interest in emotional impact), about how we embrace artifacts, devices, and products and take them into our personal world. The phenomenological paradigm is about social and cultural aspects of interaction, interaction involving our whole bodies and spirits.

6.3.3.1 Making meaning

The metaphor of interaction within the phenomenological paradigm is a form of meaning making (Harrison et al., 2007) in which an artifact (device, product, or system) and its context are mutually defining and subject to multiple interpretations by designers, analysts, users, and other stakeholders.

Use, usage, usability, usefulness, and functional descriptions all refer to what you can do with a product. Phenomenology is about what the product means to the user.

Meaning and meaning construction are central to the phenomenological view of interaction. Meaning is constructed on the fly, often collaboratively, in context and situations. "Interaction is an essential element in meaning construction" (Harrison et al., 2007). Information is interpreted through

viewpoints, interactions, histories, and local resources. Meaning and knowledge are very much local to (strongly situated in) the person who is the user and the interaction. Because knowledge and meaning are “situated” or “embodied,” both are influenced, even defined or constructed, by where you are and what you are doing, especially socially. This is why there are many contexts and, therefore, many perspectives and many interpretations versus one correct understanding and one correct set of metrics.

6.3.4 All three paradigms have a place in design and development

Consider, as an example, the design of an automobile.

Engineering view. Emphasis in the engineering view is on functionality, features, and reliability. Importance is given to performance (speed and acceleration) as well as fuel economy.

Also in the engineering view, human factors and ergonomics are paramount. For example, steering wheel thickness must fit an average human’s hand size and strength. Same for seat height, the fit of the curve on the seat to the human lower back, and safety restraints.

Human-information processing view. The presentation of critical information needed for driving must meet the limits of human signal detection. Information displays and interaction devices must support cognitive and decision making actions.

Phenomenological view. In contrast, the phenomenological view of car design is about how a car can become an integral part of the owner’s lifestyle. It’s about appeal and coolness of the ride, the joy of driving, possibly the thrill of speed, and pride of ownership.

6.4 FUN INTERACTION AT WORK

6.4.1 What about Fun at Work

Emotional impact factors such as fun, aesthetics, and joy of use are obviously desirable in personal use of commercial products, but what about in task-oriented work situations? Here usability and usefulness aspects of user experience are obvious, but the need for emotional impact is not so clear.

6.4.2 Fun Can Make Some Work More Interesting

However, there is evidence that fun can also help at work to break monotony and to increase interest and attention span, especially for repetitive and possibly boring work such as that performed in call centers. Fun can

enhance the appeal of less inherently challenging work, for example, clerical work or data entry, which can increase performance and satisfaction (Hassenzahl, Beu, & Burmester, 2001). It is easy to see how fun can lead to job satisfaction and enjoyment of some kinds of work.

Emotional and rational behaviors play complementary roles in our own lives such that emotional aspects of interaction are not necessarily detrimental to our reasoning processes for doing work. For example, software for learning, which can otherwise be dull and boring, can be made more interesting with a dose of novelty, surprise, and spontaneity.

6.4.3 But Fun Can Trade Off with Usability

However, fun and usability can conflict in work situations. Too simple can mean loss of attention, and consistency can translate as boring. But less boring means less predictable and less predictable usually goes against traditional usability attributes, such as consistency and ease of learning (Carroll & Thomas, 1988). Fun requires a balance: not too simple or boring, but not too challenging or frustrating.

6.4.4 Fun Is Not Compatible with Some Work Situations

Some work roles and jobs are simply not amenable to fun as part of the work practice. Consider a job that is inherently challenging and that requires full attention to the task, such as air traffic control. It is essential for air traffic controllers to have no-nonsense software tools that are efficient and effective. Any distraction due to novelty or even slight barriers to performance due to clever and “interesting” design features will be hated and could even be dangerous. For this kind of work, users often want less mental effort, more predictable interaction paths, and more consistent behavior. They especially do not want a system or software tool adding to the complexity.

Certainly the addition of a game-like feature is welcome in an application designed primarily for fun or recreation, but imagine an air traffic controller having to solve a fun little puzzle before the system gives access to the controls so that the air traffic controller can help guide a plane heading for a mountain in the fog.

Waterfall Lifecycle Process

One of the earliest formal software engineering lifecycle processes; an ordered linear sequence of lifecycle activities, each of which flowed into the next like a set of cascading tiers of a waterfall (Section 4.2).

6.5 WHO INTRODUCED THE WATERFALL MODEL?

It’s unclear who gets the credit (or blame) for first expounding on the waterfall model for software development. Royce (1970) is the most frequently cited source and probably the first formal description of the process. By association with that publication, Royce is credited with “inventing” or introducing the

process. But this is a misconception as the method had already been in use for some time and, it seems, the Royce paper was intended as a critique of the method as one that didn't work. And, by the way, it wasn't called the "waterfall model" until 1976 (Bell & Thayer, 1976).

And the Royce paper was, in fact, preceded (by some 14 years) by a presentation by Benington (1956) that described it as a method used to develop software for the SAGE² project. This paper was later republished (Benington, 1983) in a historical perspective, and Benington added a foreword stating that the process wasn't intended to be used in a strict top-down way and that it depended on a prototype stage. In 1985, the Department of Defense decided to make the waterfall approach a software development standard (Department of Defense, 1998) for their software development contractors.

Because the waterfall process was so organized and systematic compared with any previous approach, you might have called it the Frederick Taylor (Section 6.3.1) movement of software engineering.

6.6 SILOS, WALLS, AND BOUNDARIES

The boxes of the waterfall model (Fig. 4-1) were sometimes called "silos" because they strongly compartmentalized the lifecycle activities, a compartmentalization that was usually reflected in the developer or contractor's organization. One group or department had sole responsibility for requirement specifications and nothing else. A different department did design while a completely separate department was tasked with implementation.

6.6.1 Working in Silos

Each group working within its own box was typically organized under hierarchical administrative authority. These departmental organizations were referred to as "silos" because of their vertical organizational structure and their isolation from the other departmental groups. Requests for major decisions bubbled up to the top of each silo and the decisions filtered back down to the troops on the ground.

Rather than being inclined to collaborate in decision making, groups were more likely to be motivated to compete against each other. Worse, they couldn't

²The Semi-Automatic Ground Environment, a 1950s system of large computers and the associated NORAD cold war computer network that coordinated data from many radar sites and processed it to produce a single unified image of the airspace over a wide area (https://en.wikipedia.org/wiki/Semi-Automatic_Ground_Environment).

collaborate if they wanted to because silos make it impossible to have a shared understanding of the big picture.

It's not that a silo-based approach implies a lack of competence. Each group does its thing really well. Inside a silo you can find well-sharpened skills and even close communication. But that communication cannot easily penetrate silo walls. When these organizations started forming multidisciplinary cross-disciplinary teams, with members representing each silo, it turned out to be nearly impossible to make timely decisions about important issues. The representatives had to take each major issue back to their respective groups, then up and down the silo to get a decision, and finally return to the composite team.

And each silo group will find any desire to be innovative overcome by a motivation to protect itself against risks and fear of being blamed for failure. Fear of failure is justified because there is so much going on in the other silos that designers have no control over. It is only natural to say “no” to any suggestions to be innovative and stray from run of the mill, almost guaranteeing the real risk of a mediocre overall design at best.

Silo-oriented organizations encourage people to rally around project and administrative boundaries rather than rallying around the products or systems they are supposed to be developing. They also hide behind platform boundaries. For example, because mobile apps require different talents for analysis, design, implementation, and even marketing, it's easy for separate business, product, and engineering silos to emerge for your mobile development. This will, of course, lead to a chasm of disconnect between the mobile, desktop, and web versions of your app.

Can you expect high morale in such an environment? No. Instead of engagement, excitement, and energy, you will usually see finger pointing, disputes about ownership, poor communication, blocked collaboration, and duplication of effort on some things while other things fall through the cracks. You will usually see people “just doing their jobs” in their own specializations. If you add in any geographic distribution of the team, even in different buildings, you have completely killed any chance of being able to respond to change as it occurs during the process.

6.6.2 Throwing it Over the Wall

Work in any box of the lifecycle couldn't begin until they received the output documents from the previous box. So, for example, nothing could happen in the design group until the requirements group completed their work and “threw the documentation over the wall” from their silo to the next.

And when is it time to throw it over the wall to the group doing the next phase? It's easy: just look at the calendar. When the scheduled end date for the

current phase arrives, you are done. All unresolved problems are also given to the next silo and the former owners of these problems are quite happy they will never see them again. The walls or boundaries are the hand-off points for both documentation and responsibility. This putting off of harder problems can continue until, at some point, it is painfully obvious that something is badly wrong. You have something critical that needed to be fixed a long time ago. Now it's time to see what the project manager is made of.

6.6.3 Many Projects Collapsed Under the Weight

In the software engineering (SE) world, the waterfall model has proven to be difficult in practice, at least for larger and more complex systems. Because this kind of top-down lifecycle process is very thorough and features a full system overview upfront, it was said that these systems were designed holistically and that sounds like a good thing. But, in fact, this long and drawn-out lifecycle has proven to be unsustainable in practice, resulting in projects that are over budget, behind schedule, can't manage change, or just plain failed.

6.6.4 UX Design Suffers

A marketing manager might understand the needs that get expressed as requirements and a business analyst might own the resulting features, but neither has anything to do with creating the design. Today, a UX specialist might create the interaction design, but in this kind of development environment, a software engineer or programmer or web developer will change that design as needed to put it together in the software, undermining the UX design and creating the real, but not always satisfying, user experience.